CAMPUS LOST & FOUND

A web application for reporting and reclaiming lost items on campus

Synopsis of the major project submitted by [Student name] ([USN]) and [Student name] ([USN])
Under the guidance of [Guide name], Department of Computer Science and Engineering
Sample Institute of Technology · Academic year 2026-27

1. Abstract

Students lose ID cards, calculators, bottles and chargers every week, and the usual ways of finding them (a notice on the board, a message in a class WhatsApp group) rarely reach the right person. This project builds a web application where anyone on campus can post a lost or found item with a photo, search posts by category and place, and claim an item by answering a question that only the real owner could answer. The person who posted the item sees the answer, approves or declines the claim, and arranges the hand-over through messages inside the app. A campus office account can hide posts that break the rules. The system is built with HTML, CSS and JavaScript on the front end and a REST API in Node.js with Express and SQLite on the back end.

2. Literature survey

Most colleges handle lost items in one of three ways. A notice board or a security-desk register works only for people who walk past and read it. Class WhatsApp groups reach more people, but messages are buried within hours and cannot be searched by item or place. General survey forms collect reports in one sheet, but nobody is notified and there is no way to verify ownership. Dedicated lost and found systems used by airports and large venues add search and claim verification, but they are run by staff, not by the people who found the item. This project combines a searchable shared list with owner verification done by the finder, which suits a campus where most items are found by other students.

3. Problem statement

There is no single, searchable place on campus to report lost and found items. Notices go unread, group messages get buried within hours, and when someone does reply "that's mine", the finder has no way to check the claim before handing the item over.

4. Objectives

1. Let students post lost and found items with a photo, category, place and date.
2. Make posts searchable and filterable so a matching item can be found in seconds.
3. Verify ownership with a proof question set by the finder before any hand-over.

SAMPLE

Sample document by Final Commit for a fictional college. Names in square brackets are placeholders.

4. Keep the conversation about the hand-over inside the app, between the two people involved.
5. Give the campus office a way to moderate posts.

5. Modules

Module	Description
Authentication	Login with college email and password; passwords stored as salted scrypt hashes; session tokens.
Posts	Create lost/found posts; photos resized in the browser before upload; status open, claimed or returned.
Search	Filter by type, category and place; keyword search on title, description and place.
Claims	Owner answers the proof question; poster approves or declines; one claim per person per item.
Messages	Conversation between the poster and the approved claimant only.
Moderation	Campus office can hide and unhide posts.

6. Requirements

Software	Hardware
Node.js 24, Express 5, SQLite (node:sqlite), HTML5, CSS3, JavaScript (ES modules), Git, VS Code, any modern browser	Any laptop with 4 GB RAM; the server also runs on a free cloud tier

7. Expected outcome

A working web application that a college can host, in which a lost item can be reported, found, verified and returned without a notice board or group message, along with the project report, test results and a user guide.

8. Timeline

Weeks	Work
1	Requirements, synopsis, database design
2 to 3	Authentication, posts and search
4	Claims, messages and moderation
5	Testing, deployment, report and presentation

9. References

1. R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, University of California, Irvine, 2000.

SAMPLE

2. OWASP Foundation, "Password Storage Cheat Sheet." https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
3. OWASP Foundation, "SQL Injection Prevention Cheat Sheet." https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
4. OpenJS Foundation, "Express documentation." https://expressjs.com/
5. OpenJS Foundation, "Node.js documentation: SQLite (node:sqlite)." https://nodejs.org/api/sqlite.html

SAMPLE